poi-agent一周学完agent

这个tutorial能干什么

一周(5天 * 8小时)学完agent，并且能找到agent相关实习！

告诉你 Agent 其实真的非常简单！很多人学不会是因为踩坑（例如学了langchain）或绕远路了！

第一部分：学会 Agent 基础（学习需约 1天 * 8小时）

1. 拥有你自己的llm api-key（阅读需约15分钟）
   - 你可能需要学会使用python和uv 用 rust 编写，类似于 rust 里面的 cargo，非常非常快
   - 为什么不用conda而是uv:uv 开源无商用风险，Conda 在超过 200 人的组织有潜在商用授权问题
   - uv 可以像 pip 一样编辑镜像源，mac 和 linux 系统修改 `~/.config/uv/uv.toml` 并写入类似于下面的内容，Windows 系统可以自己查下怎么配置。项目初始化需要 `uv sync` 。

   代码块

   ```
   [[index]]
   url = "https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple"
   default = true
   ```

   - 你可能需要llm api-key，推荐kimi或者智谱
   - 配置环境变量 `OPENAI_API_KEY` 和 `OPENAI_BASE_URL` ，并且尝试运行 `core/llm.py`
2. 实现 Node / Workflow / Agent （阅读需约2小时）
   - 我们最终的目标是造一个Agent，能够联网搜索、运行命令行、文件编辑。
   - 我们需要用到例子 https://github.com/lasywolf/poipoi-agent
   - Agent底层可以使用Node来抽象，我已经准备好了一个极简的实现，可以看 `core/node.py` ，不到60行就实现了一个Agent的轻框架，实在是太容易理解了！如果没有py基础看不懂，可以把代码复制给ai让它来解释。
   - 但我们应该怎么去用Node呢，我们可以新建3个Node并把它们串起来实现功能 `接收输入->上网搜索->大模型生成总结` ，恭喜你已经实现了workflow，相关实现已经在 `examples/workflow`

- 现在新建个workflow，实现功能 `接收用户输入->大模型回复` ，并且loop反复调用这个workflow，恭喜你已经实现了chatbot，相关实现已经在 `examples/chatbot`
- 现在尝试给chatbot一些tools(下文会详细讲tool)，让它能够上网搜索东西、编辑文件、运行命令行，恭喜你已经实现了agent，相关实现已经在 `examples/chatbot_with_tools`
- 总结：**workflow = node + node**、**chatbot = workflow + loop**、 **agent = chatbot + tools = workflow + loop(循环) + tools**
- 恭喜你已经懂Agent了！可能你现在会有疑惑“这么简单的一个轻框架能有用吗，为什么选择自己造轻框架/PoiFlow/PocketFlow等等 而不是 LangChain / Dify / Coze / Google ADK / Spring AI”，并且我并没有看到过有人用langchain开发出来好产品，以及langchain还出过非常严重安全漏洞 CVE-2025-68664，以及存在过度抽象、依赖地狱、bug多、不灵活难以定制等问题。事实上优秀的agent都是采用自己搭建轻框架来开发的，例如claude-code、cursor、kimi-cli、pi-mono、Pocketflow-examples等等，所以非常推荐自己搭建轻框架或者直接调用llm api来开发。

3. 实现 RAG （阅读需约1小时）
   - 选择 Chroma 而不是Milvus、LanceDB、pgvector等等，因为Chroma部署简单、api简洁使用成本低
   - 注意，你可能需要一个 embedding model api 而不是 llm api，你可以在智谱等官网找到对应的 embedding 模型
   - 恭喜你已经学会了rag！！！
   - 为什么这就是全部了，真有这么简单？是的这就是全部，当初提出 RAG(Retrieval-Augmented Generation) 概念时，可能觉得得有 检索-增强-生成 这三个功能。
   - 但实际上大伙最终只用到了检索，VectorDB 就能很完美执行这个任务
   - 所以 RAG 是个很过时的概念，大伙只想要一个 VectorDB 而已，或者说 RAG=VectorDB
4. 实现 Tool、MCP、Skill （阅读需约1小时）
   - Tool: process call (function call)，也就是调用了一个函数
   - MCP: remote process call 也就是后端里面的 RPC 概念，调用了服务器上的一个函数
   - Skill: local process call 也就是调用了本地的一个函数
   - 它们其实本质上都是 Tool
   - 为什么Tool会有这么多形式？这就不得不说Tool的发展历程。
   - Tool来源：在早期大伙为了chatbot不只是chat，而是实际做些事所以出现了Tool，实现Tool的形式各不一样，最常见的就是输入llm的prompt中里面加入function（等同于tool）的name、parameters、description并且要求llm输出json格式，最后再调用对应的function。

- MCP来源：为了解决Tool实现参差不齐的现象，anthropic定了一个Tool的标准，也就是MCP(Model Context Protocol)（感觉不如叫Remote Tool Protocol更易读），让各个Tool能够远程“即插即用”，看起来非常棒只要提供了MCP服务就能实现ai从just chat到do something的转变，于是2025年各个公司疯狂都在推行自己的MCP服务
- Skill来源：但人们渐渐发现了MCP的弊端：每次调用llm时候，都会把在prompt额外加上MCP的所有Tool的信息（包括name、parameters、description等等），发现大部分MCP服务并没有想象的那么有用，以及导致性能变差以及token浪费，anthropic在blog描述了这件事 Code execution with MCP: Building more efficient agents，并分享了它们的解决方案，就是**渐进式加载**和**多用代码执行**，后来anthropic发布了skill就和这个差不多，重点就是**渐进式加载**和**多用代码执行**。
- 设计Tool：实际Agent并不需要那么多五花八门的Tool，最重要的是linux中的bash、edit、find、grep、ls、read、write命令，这些就已经能做很多事且做得非常好，Vercel通过移除大部分的Tool反而提高了text-to-sql从80%到100%，以及pi-mono极简coding-agent作者提到这四个工具就是构建有效 Coding Agent 所需的全部：read、write、edit、bash
- 实践：可以阅读 `tools` 和 `examples/chatbot_with_tools` 文件夹里的实现
- 总结: MCP是Remote Tool，Skill是Local Tool，尽量不要设计Tool并且优先用linux的bash来解决问题

5. 实现 kimi-cli 或 pi-mono （阅读需约1小时）
   - 你为什么应该实现这个 coding-agent 项目，而不是智能客服项目(dify + rag)，找实习的同学可以把这项目写简历上很好用
   - 因为 coding-agent 项目足够复杂去体现你的 agent 能力，且 coding-agent 做万物就是未来趋势
   - 推荐kimi-cli和pi-mono因为他们开源且实现不错，推荐看 blog What I learned building an opinionated and minimal coding agent
6. 实现 Context / Memory（阅读需约1分钟）
   - 短期 Context：最近几轮的完整对话
   - 长期 Context：更早的对话，会"总结一次"进行压缩
   - Memory = 短期 Context + 长期 Context
7. 实现 Multi-Agent / Subagent / Agent Teams （阅读需约1小时）
   - multi-agent最初设想用google制定的A2A(agent to agent)协议，让不同地方的Agent进行交互，但这个设想失败了，multi-agent效果复杂且大部分性能还不如简单的single agent，且现实中没看到过agent用A2A协议进行交互
   - 但大伙发现有些场景可以用multi-agent来实现上下文隔离、只回传压缩结果、避免主上下文被工具细节污染，这样能提高agent的效果，可以看这个blog了解multi-agent到底是什么How we built our multi-agent research system

- subagent概念由此发展出来，甚至推出了自定义subagent。但我并不推荐自定义subagent，毕竟由master agent来自动生成subagent总是个简单高效的选择
- Agent Teams则是目前最前沿的发展方向，摒弃了主从的agent结构，而采用并行的方式，能够成倍效率且agent间不冲突地开发项目，这是十分有价值的，大伙都在研究，可以参考claude的agent-teams以及blog Building a C compiler with a team of parallel Claudes 还有cursor的blog 扩展长时间运行的自主编码能力 和 迈向自动驾驶代码库

第二部分：达到实习要求 （学习需约 4天 * 8小时）

1. 了解面试都会问什么，以及项目推荐（阅读需约1小时）
   - agent 面试只问项目，没有八股
   - 问你的项目，推荐实现 coding-agent 项目 也就是 pi-mono 或 kimi-cli（可能需要4天时间来完成）
   - 可能会问以下内容，所以你需要知道整个流程运转（但不一定要实践出来）：Agent 效果可评估、提示词自动优化、Agentic Sandbox、Agent Teams
   - 推荐编写简历的开源免费网站rxresu.me
   - 已经足够去实习面试了
2. 关注 Agent 效果可评估 与 提示词自动优化（阅读需约1小时）
   - agent 效果可评估（较难）
   - 提示词自动优化的设计思路和上面这个 blog 差不多，需要设计指标并观测
3. 关注 Agent Teams（阅读需约1小时）
   - Agent Teams 概念 以及 blog Building a C compiler with a team of parallel Claudes
   - 扩展长时间运行的自主编码能力（较难） 和 迈向自动驾驶代码库（较难）
5. 了解 Sandbox（阅读需约1小时）
   - 用过 docker 就行了，使用docker作为sandbox可以应对绝大多数场景
   - 部分对延时要求非常高的场景，需要做专门的agentic infra来优化延时（太复杂了了解下就好）
   - 推荐阅读 为本地代理实现安全沙箱